

LLM-Based AI Agents and Multi-Agent Systems

AI Uncovered: The LLM and Agents Masterclass 2025

Session By:



Tezan Sahu
SWE 2
BizChat Extensibility, E+D



Gurparkash Singh
Data & Applied Scientist 2
STCI



Shubhojit Mallick
Senior Applied Scientist
STCI

Agenda

- Introduction to LLM-Based AI Agents
- The Rise of LLM-Based AI Agents
- Automation vs Workflows vs AI Agents
- Current State of AI Agents
- Memory in AI Agents
- The AI Agent Stack
- Agentic Design Patterns
- Introduction to Multi-Agent Systems (MAS)
- Common Agent Roles in MAS
- Agent Communication in MAS
- Live Demo(s)
- Observability in Multi-Agent Systems
- Designing Resilient AI Agents
- [Optional] Computer-Using Agents (CUA)

Introduction to LLM-Based AI Agents

- **What are AI Agents?**

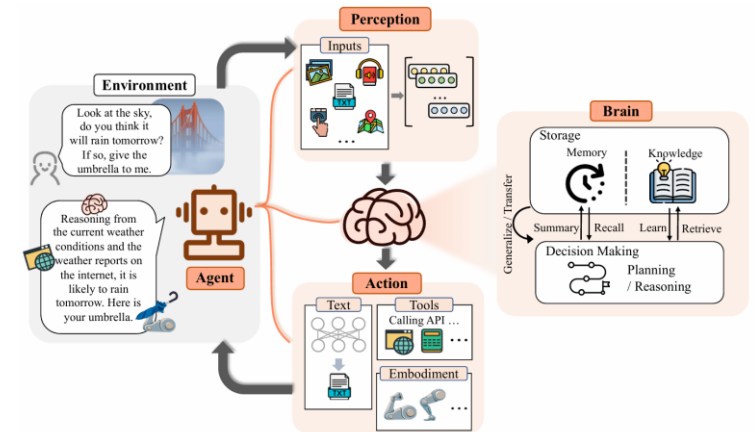
- Entities that **perceive** their environment, **reason**, and **act** (autonomously) to achieve **specific goals** by designing its workflow (**planning**) and utilizing available tools (**execution**)

- **The Old Way: Traditional Agents**

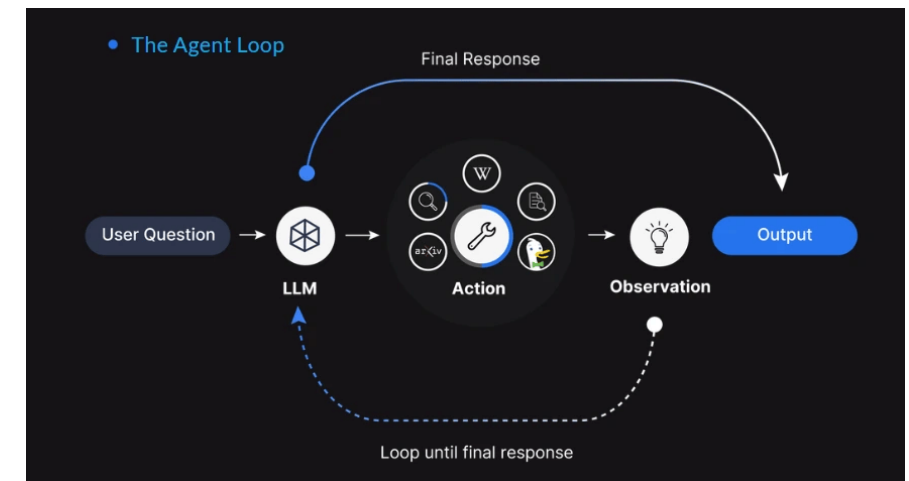
- **Rule-Based:** Heavily reliant on hand-crafted rules (IFTTT)
- **Limited Scope:** Brittle and unable to handle tasks outside their pre-defined logic
- **High Maintenance:** Required significant effort to build and adapt

- **The Paradigm Shift: LLM-Powered Agents**

- **The "Brain":** Large Language Models (LLMs) act as the core reasoning engine
- **Core Abilities:** LLMs provide advanced capabilities in understanding natural language, planning complex actions, and synthesizing information.
- **Flexibility & Adaptability:** Can generalize from examples and handle novel, unseen tasks with remarkable proficiency.



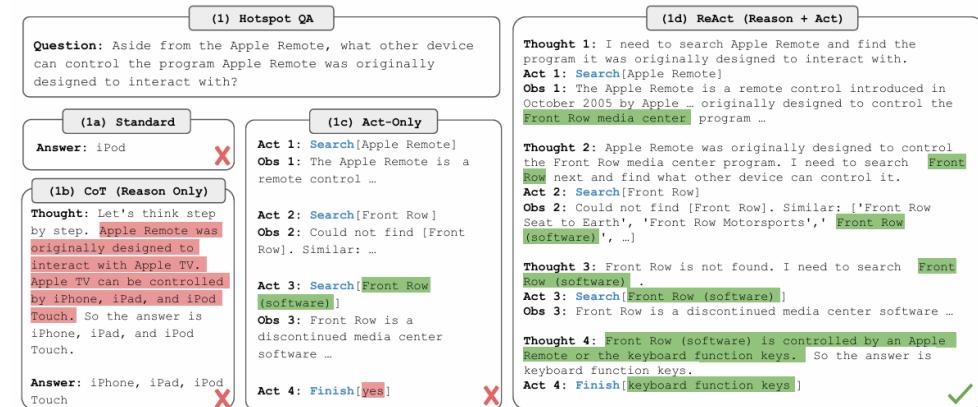
Ref: [The Rise and Potential of Large Language Model Based Agents: A Survey](#)



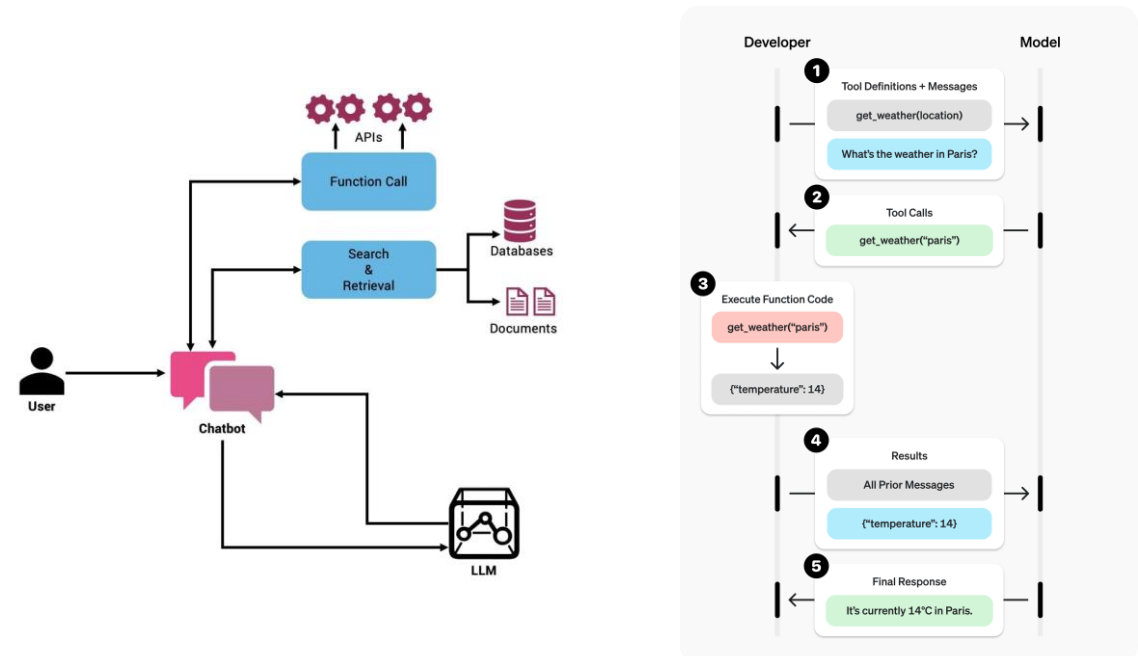
Ref: [Analytics Vidhya - Top 10 AI Agent Trends and Predictions for 2025](#)

The Rise of LLM-Based AI Agents

- Function-Calling in LLMs
 - ReAct (Reason + Act) Pattern (Oct 2022)
 - Introduced the use of "Thought" and "Action" tokens for language-based tool invocation.
 - Used this pattern with external tools like Wikipedia search and calculators.
 - Demonstrated that LLMs can **decide what tool to invoke, pass arguments, and use outputs**, all via natural language.
 - Toolformer [Ref] (Feb 2023)
 - Trains LLMs to autonomously use tools by self-generating labeled data
 - Helped establish tool use as a training objective
 - OpenAI Function Calling API (Jun 2023)
 - Significant developer-facing standard for function calling
 - Allowed structured schema-based function execution from ChatGPT (GPT-4 / GPT-3.5)

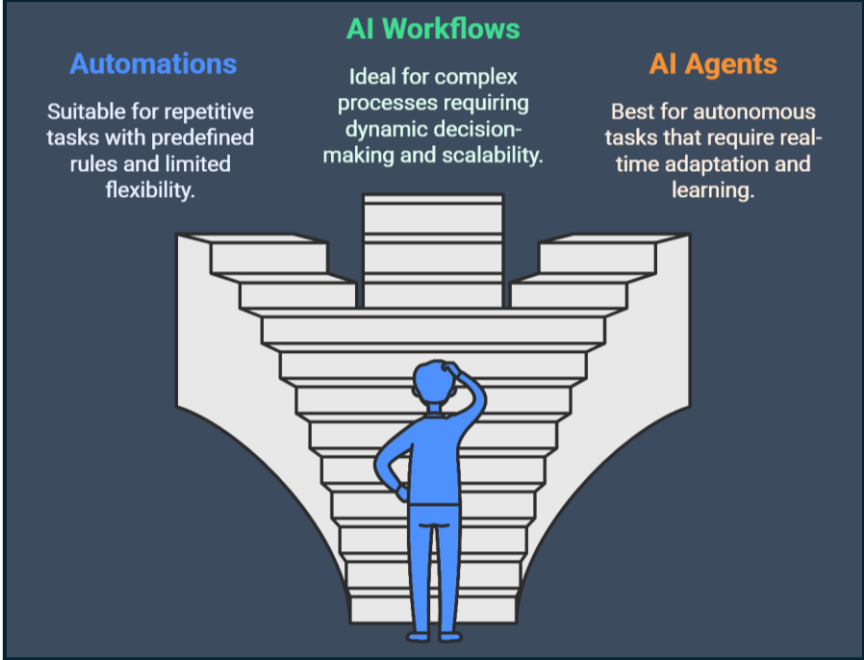


Ref: [REACT: SYNERGIZING REASONING AND ACTING IN LANGUAGE MODELS](#)



Automation vs Workflows vs AI Agents

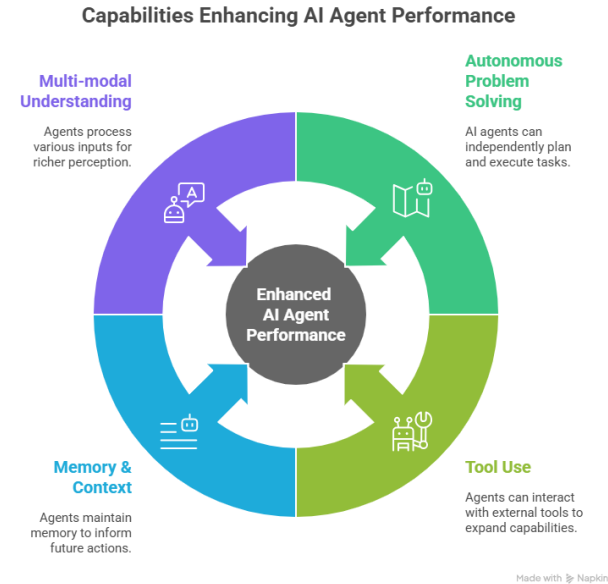
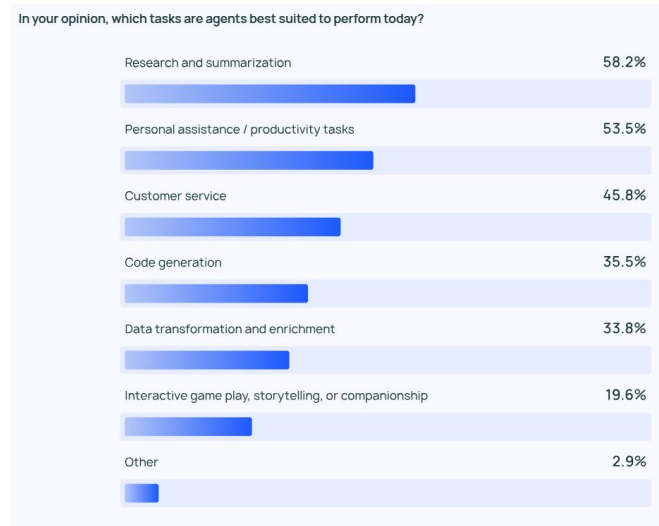
	Automation	AI workflow	AI agent
Definition	 A program that executes predefined, rule-based tasks automatically	 A program that calls an LLM via API for one or more steps	 A program designed to perform non-deterministic tasks autonomously
Core foundations	1011 0110 Boolean logic	1011 0110 Boolean logic ≡ Fuzzy logic	≡ Fuzzy logic ↻ Autonomy
Tasks	Deterministic, predefined tasks	Deterministic tasks requiring flexibility	Non-deterministic, adaptive tasks
Strengths	• Delivers reliable outcomes • Fast to execute	• Better handling of complex rules • Great for pattern recognition	• Highly adaptive to new variables • Simulates human-like behavior and reasoning
Weaknesses	• Limited to tasks explicitly programmed • Cannot adapt to new scenarios • Struggles with complexity	• Requires data to train models effectively • Harder to debug and interpret	• Less reliable, may produce unpredictable undesired outcomes • Slower to execute
Example	Send a Slack notification every time a new lead signs up on our website	Analyze, score and route every website inbound lead using ChatGPT	Perform a full internet search on every inbound lead and update infos



Ref: Automations vs AI Workflows vs AI Agents: Understanding the Key Differences by Tyler Gregory (LinkedIn)

Current State of AI Agents

- Based on Langchain's “[State of AI Agents Report](#)”:
 - 51.1% Companies have Agents in production
 - 90% Respondents from Non-Tech Companies have or plan to have Agents in production

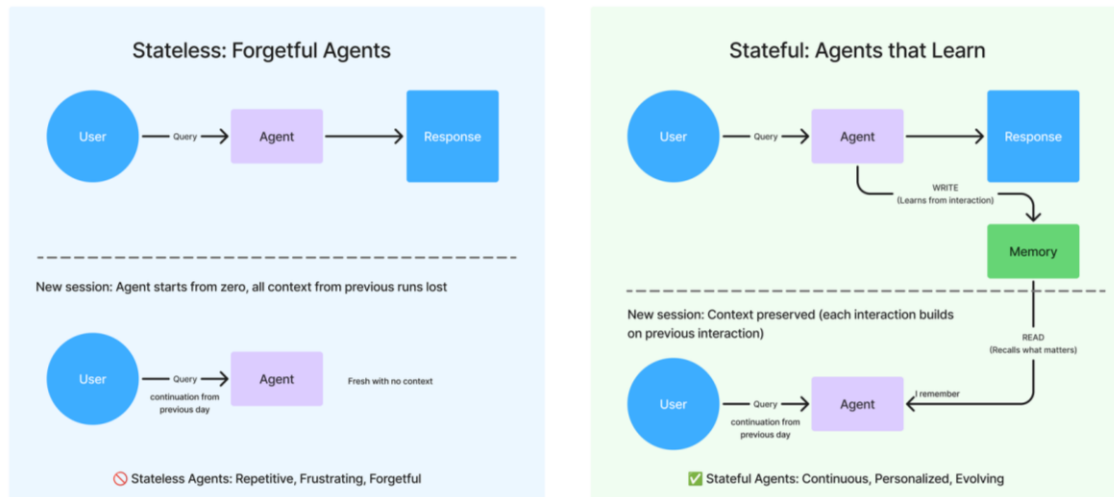


- Latest Trends
 - Computer Using Agents (CUAs)
 - Reasoning-Powered Agents (using Reasoning Models like o1, o3, DeepSeek-R1, Gemini 2.5-Pro, etc.)
 - Multi-Agent Systems

Memory in AI Agents

Memory: The Missing Layer in AI Agent Architecture

From stateless interactions to continuous intelligence



Ref: [Memory in Agents: What, Why and How | Mem0 AI](#)

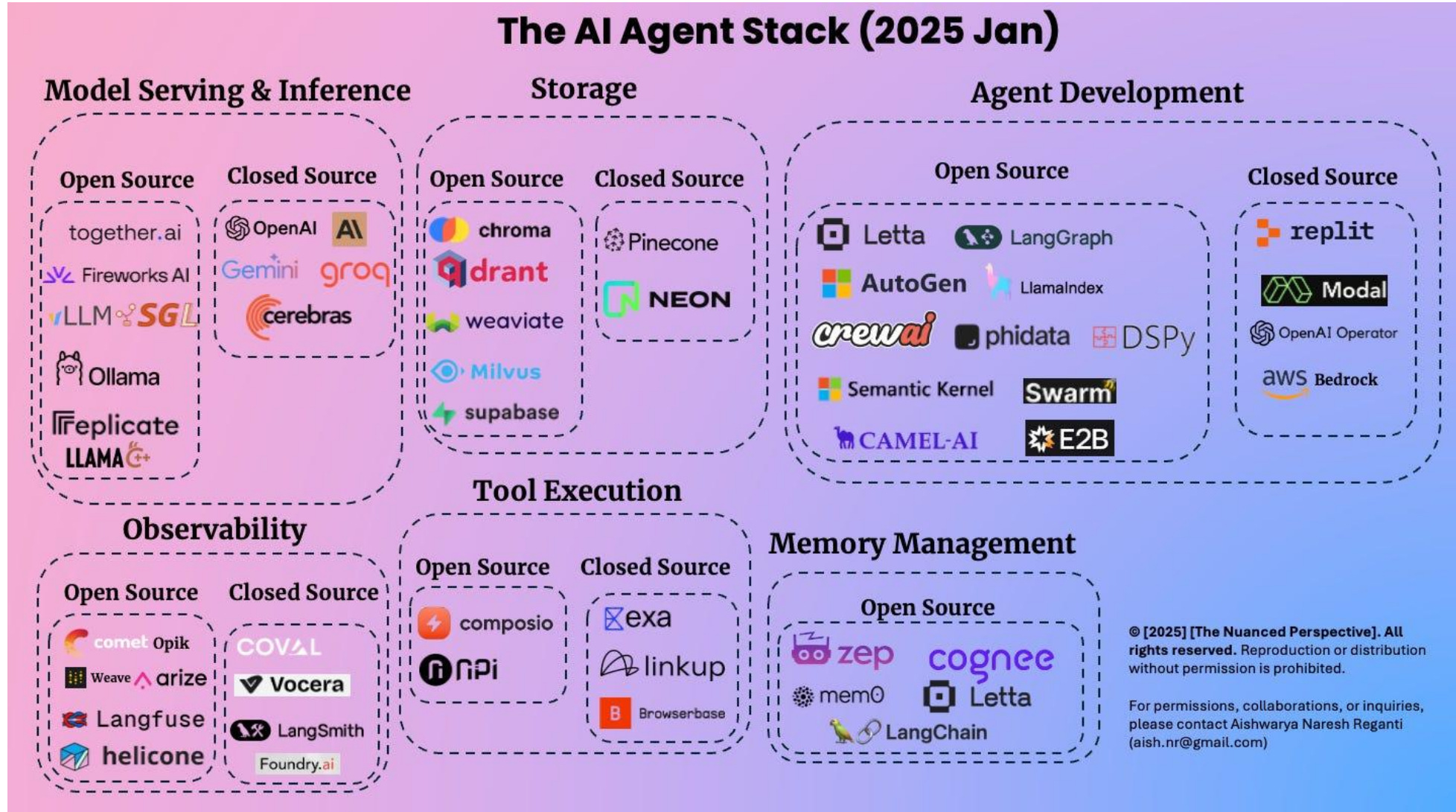
• Busting Common Misconceptions

- Context Window $\neq$ Memory
 - Context windows help agents stay consistent *within* a session
 - Memory allows agents to be intelligent *across* sessions.
- RAG $\neq$ Memory
 - RAG brings external knowledge into the prompt at inference time
 - RAG helps the agent answer better.
 - Memory brings in continuity
 - Memory helps the agent behave smarter.

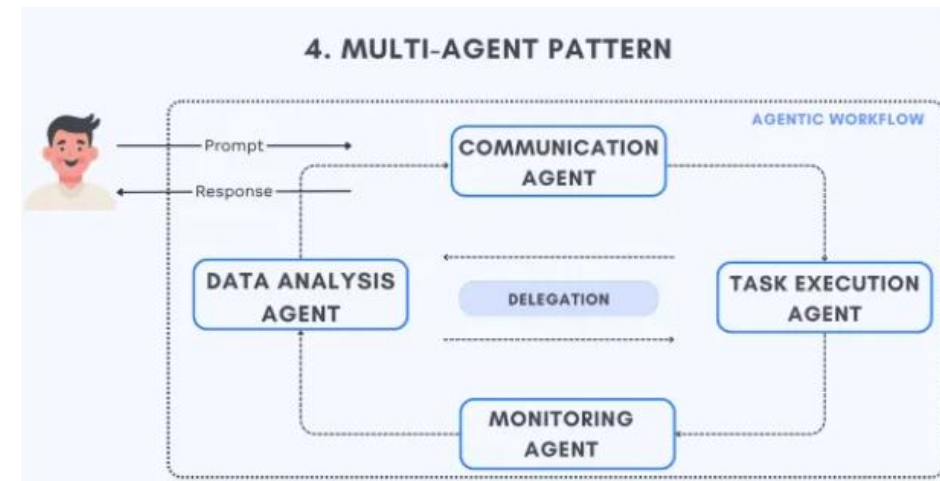
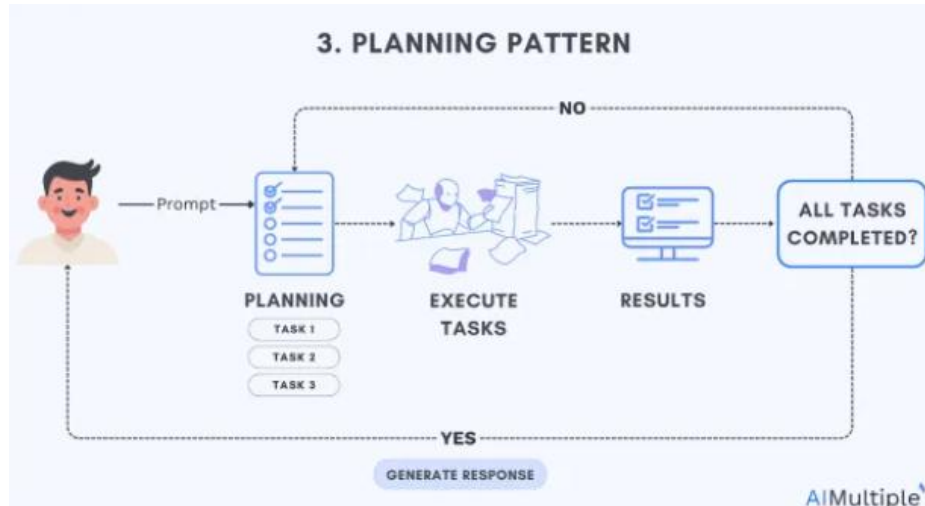
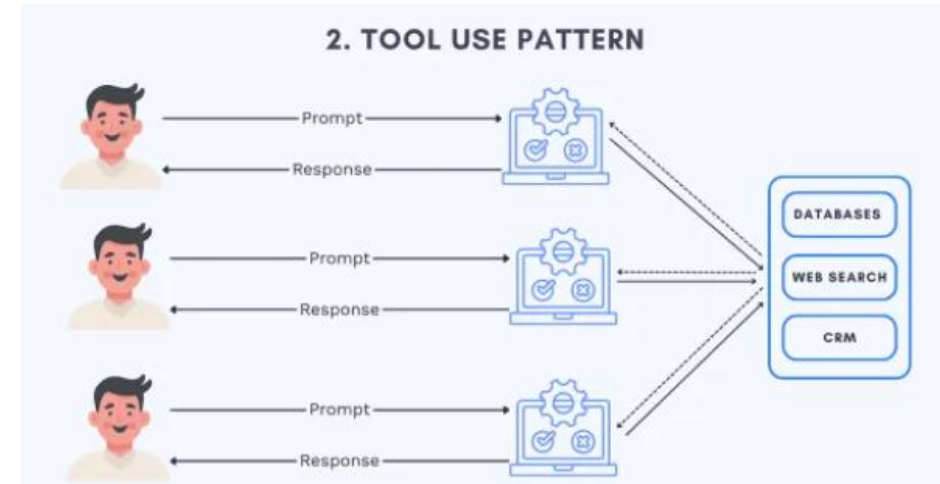
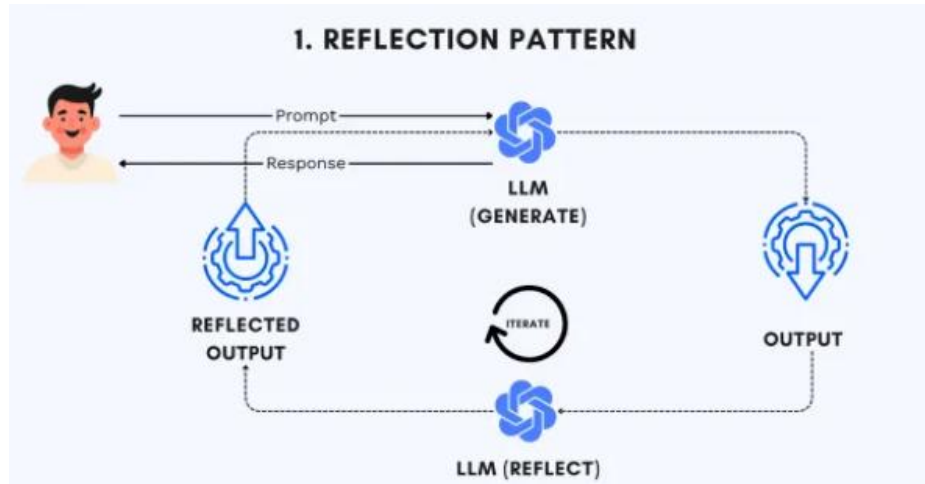
• Types of Memory in Agents

TYPE	ROLE
Working Memory (short-term)	Maintains short-term conversational coherence
Factual Memory (long-term)	Retains user preferences, communication style, domain context
Episodic Memory (long-term)	Remembers specific past interactions or outcomes
Semantic Memory (long-term)	Stores generalized, abstract knowledge acquired over time

The AI Agent Stack



Agentic Design Patterns



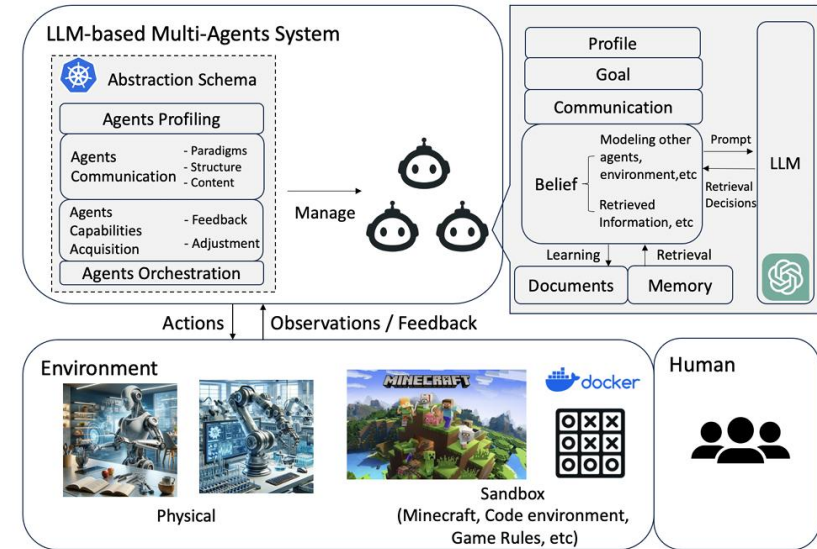
Introduction to Multi-Agent Systems (MAS)

- **What is a MAS?**

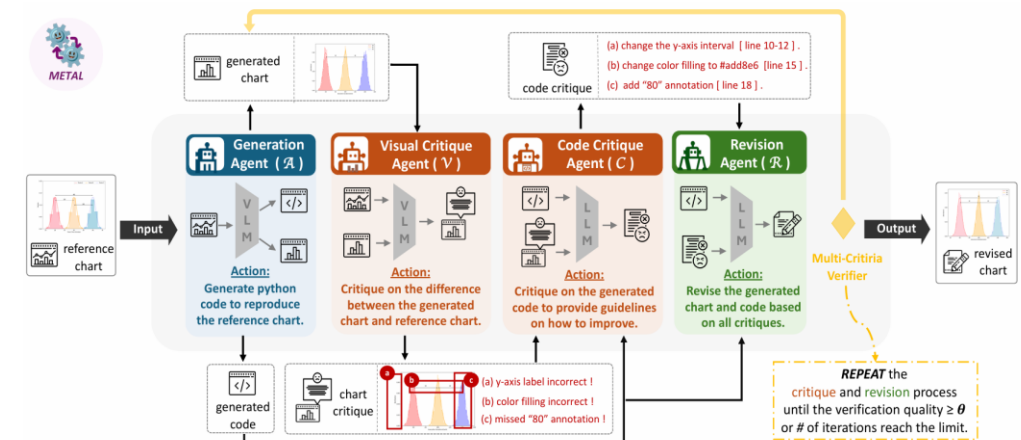
- A system composed of **multiple autonomous agents** that **interact with each other** to solve a problem that is **beyond the capabilities of any single agent**

- **Why MAS?**

- **Complexity:** Decompose a massive problem into smaller, specialized tasks.
- **Scalability:** Parallelize work across multiple agents.
- **Robustness:** The system can continue to function even if one agent fails.
- **Specialization:** Each agent can be optimized for a specific skill
 - E.g.: A “code generator” agent, a “visual critique” agent, a “code critique” agent & a “revision” agent in METAL



Ref: [Large Language Model based Multi-Agents: A Survey of Progress and Challenges](#)



Ref: [METAL: A Multi-Agent Framework for Chart Generation with Test-Time Scaling](#)

Common Agent Roles in MAS

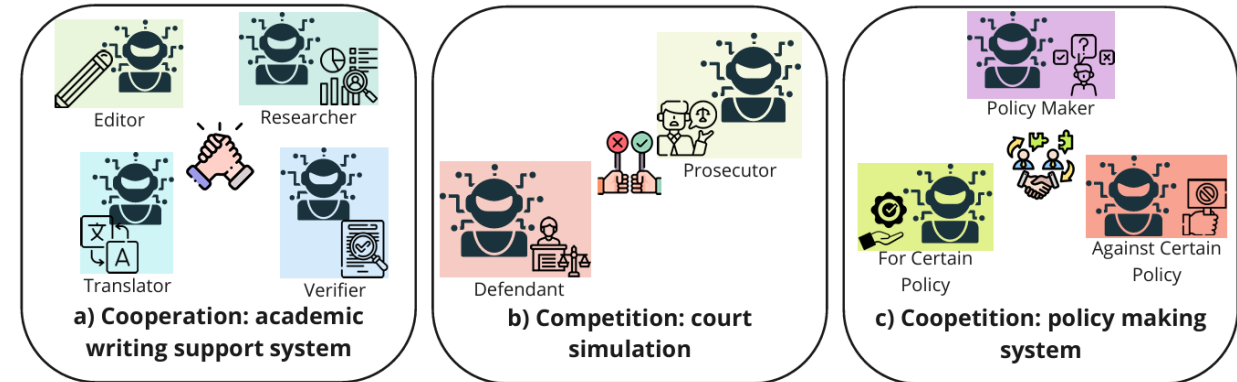
While specific domains may require specialized agents in a MAS, all of them can be broadly categorized into 4 common roles:

Role	Planner	Retriever	Executor	Critic
 Function	Task decomposition and planning	Information retrieval from sources	Task execution, API calls, writing	Evaluation and feedback provision
 Focus	Strategy	Research	Action	Quality Assurance
 Skill	Strategic thinking	Information gathering	Task completion	Evaluation skills

Agent Communication in MAS

Communication Paradigms

- **Cooperative**
 - Work together towards a shared goal
 - Exchange information to enhance a collective solution
- **Competitive**
 - Agents work towards their own goals that might conflict with the goals of other agents
- **Debate/Coopetitive**
 - Agents engage in argumentative interactions
 - Present & defend their own viewpoints, while critiquing others
 - Ideal for reaching a consensus or a more refined solution



Ref: [Multi-Agent Collaboration Mechanisms: A Survey of LLMs](#)

Agent Communication in MAS

Communication Structure

- **Centralized**
 - **Structure:** A single "coordinator" or "manager" agent has a global view of the system. It assigns tasks to other agents and synthesizes their results.
 - **Analogy:** A traditional top-down corporate hierarchy.
 - **Pros:** Simpler to design and control; can achieve globally optimal solutions.
 - **Cons:** The coordinator is a single point of failure and a potential performance bottleneck.
- **Decentralized**
 - **Structure:** Agents are peers and make decisions based on local information and direct communication with other agents. No single boss.
 - **Analogy:** A flock of birds or a colony of ants.
 - **Pros:** More robust and resilient; highly scalable.
 - **Cons:** Achieving coherent group behavior is more complex; may result in locally optimal but globally suboptimal solutions.

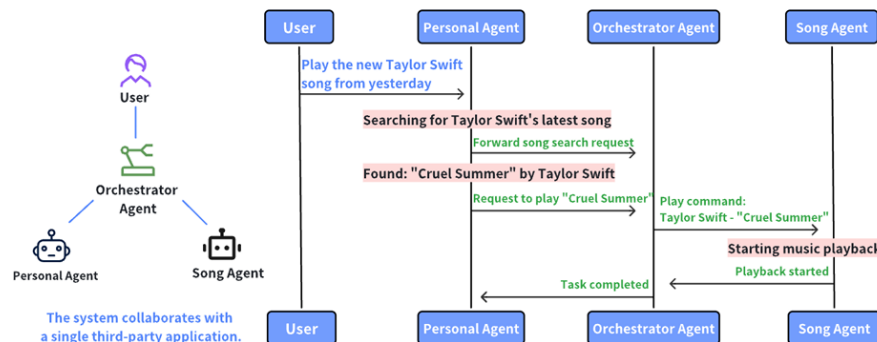


Figure 5: Centralized Architecture of LaMAS.

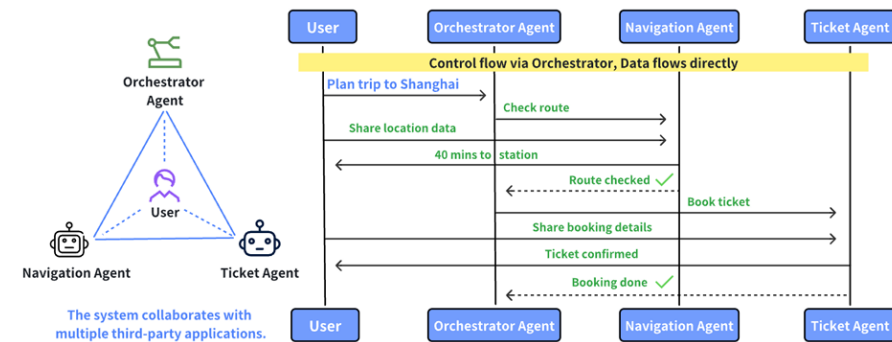


Figure 7: Decentralized Star Architecture of LaMAS.

Agent Communication in MAS

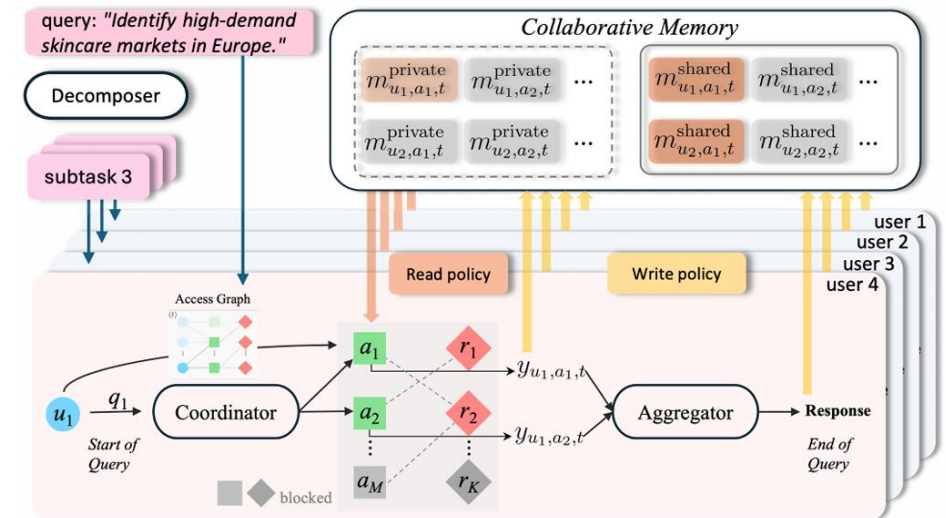
Message Passing

Agents send structured messages to one another (e.g., using JSON). This can be a direct request, a response, or a broadcast.



Shared Memory

Agents read and write to a common data space, like a "blackboard" or a shared database. This allows for indirect communication.



Ref: [Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control](#)

Time for some Live Action!

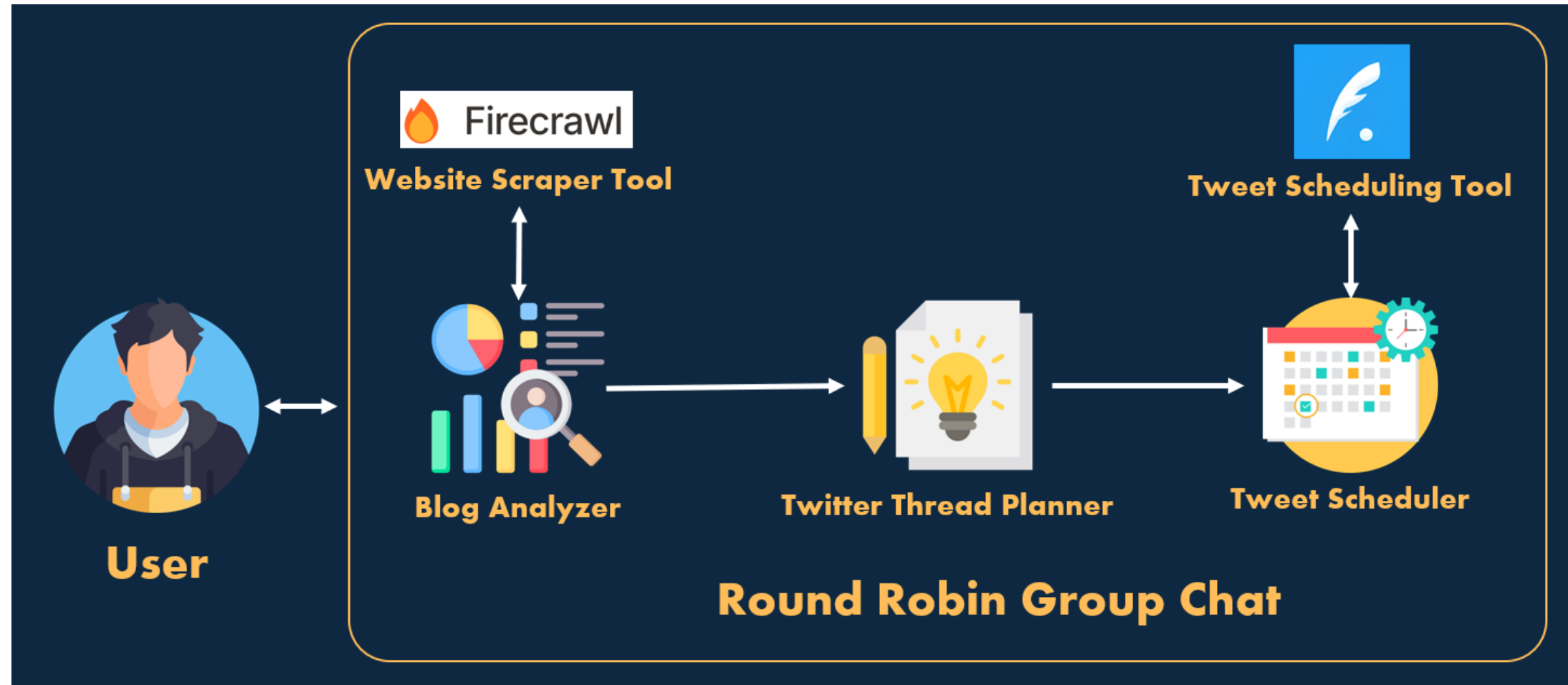


Environment Setup

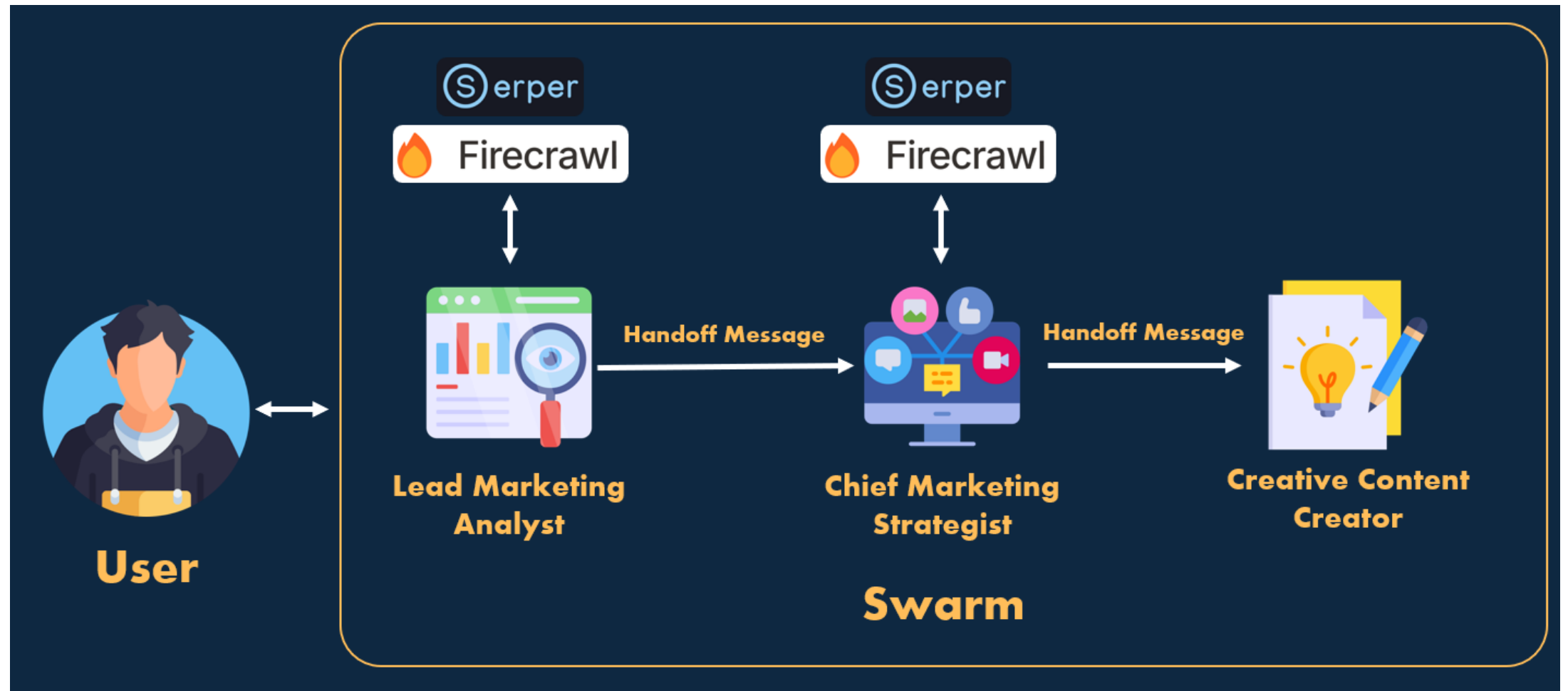
- Clone the repo: [tezansahu/building-ai-agents-with-autogen](https://github.com/tezansahu/building-ai-agents-with-autogen)
- Create Virtual Environment
- Install all required packages
- Set up GitHub Personal Access Token (PAT)
 - Access LLMs (GPT-4o, GPT-4o-Mini, etc.) for FREE
- Get API Keys for Tools
 - Serper (*for web search*)
 - Firecrawl (*for website scraping*)
 - Typefully (*for tweet scheduling*)

PS: Detailed steps for environment setup are mentioned in the [0-Environment-Setup folder of the repo](#)

Blog-to-Twitter Thread Scheduler – Sequential



Marketing Campaign Creator – Swarm



Observability in Multi-Agent Systems

- **Key Aspects of Observability**

- Transparency
 - Tracing how inputs transform into outputs & revealing the internal reasoning mechanism
- Interpretability
 - Translating complex AI decisions into human-understandable insights
- Continuous Monitoring
 - Tracking performance, detecting biases & ensuring consistent and reliable AI behavior

- **Why Observability Matters?**

- Enables trust
- Ensures reliability and efficiency
- Facilitates early detection of anomalies

- **Key Metrics to Track**

- **Performance Metrics:** Response times, latency, and throughput
- **Resource Utilization:** LLM token usage, cost
- **Error Rates:** Frequency and types of errors encountered
- **Task Completion Rates:** % of times a user's task was completed
- **User Interactions:** Engagement levels and satisfaction scores

- **Relevant Frameworks**



How to Design Resilient AI Agents?



What if an agent is stuck in a loop?

→ How would you detect and resolve infinite loops or repetitive behavior?



What if the agent does not respond within the allotted time?

→ How would you handle timeouts and ensure graceful fallback?



What if the agent loses internet connectivity mid-task?

→ Can the agent recover or retry after a network failure?



What if the agent receives incomplete or corrupted input?

→ How would you validate and clean the input before processing?

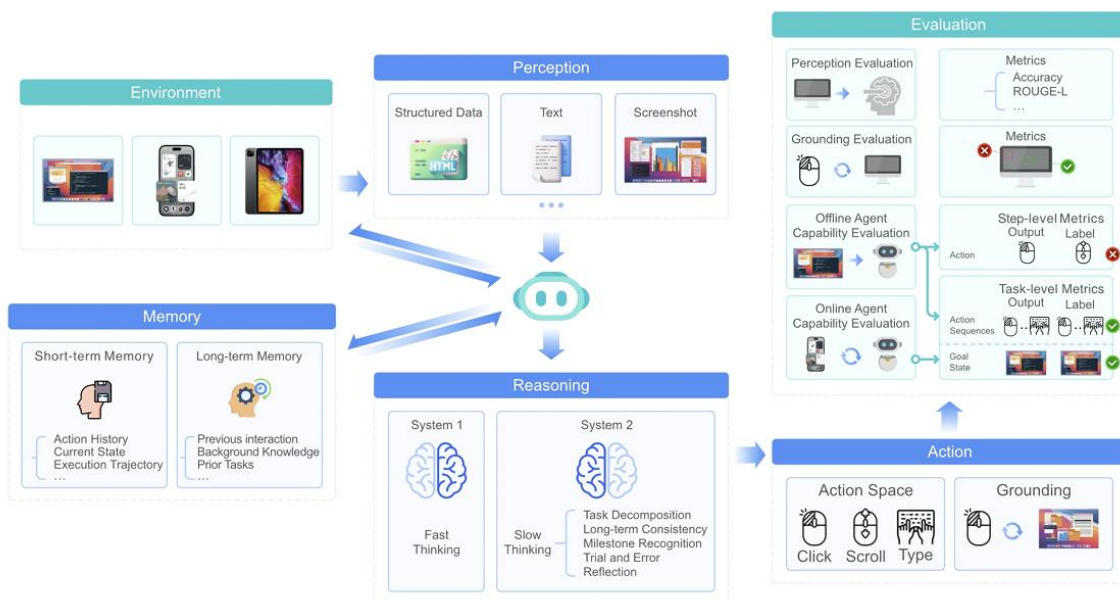


What if the agent generates sensitive or harmful content?

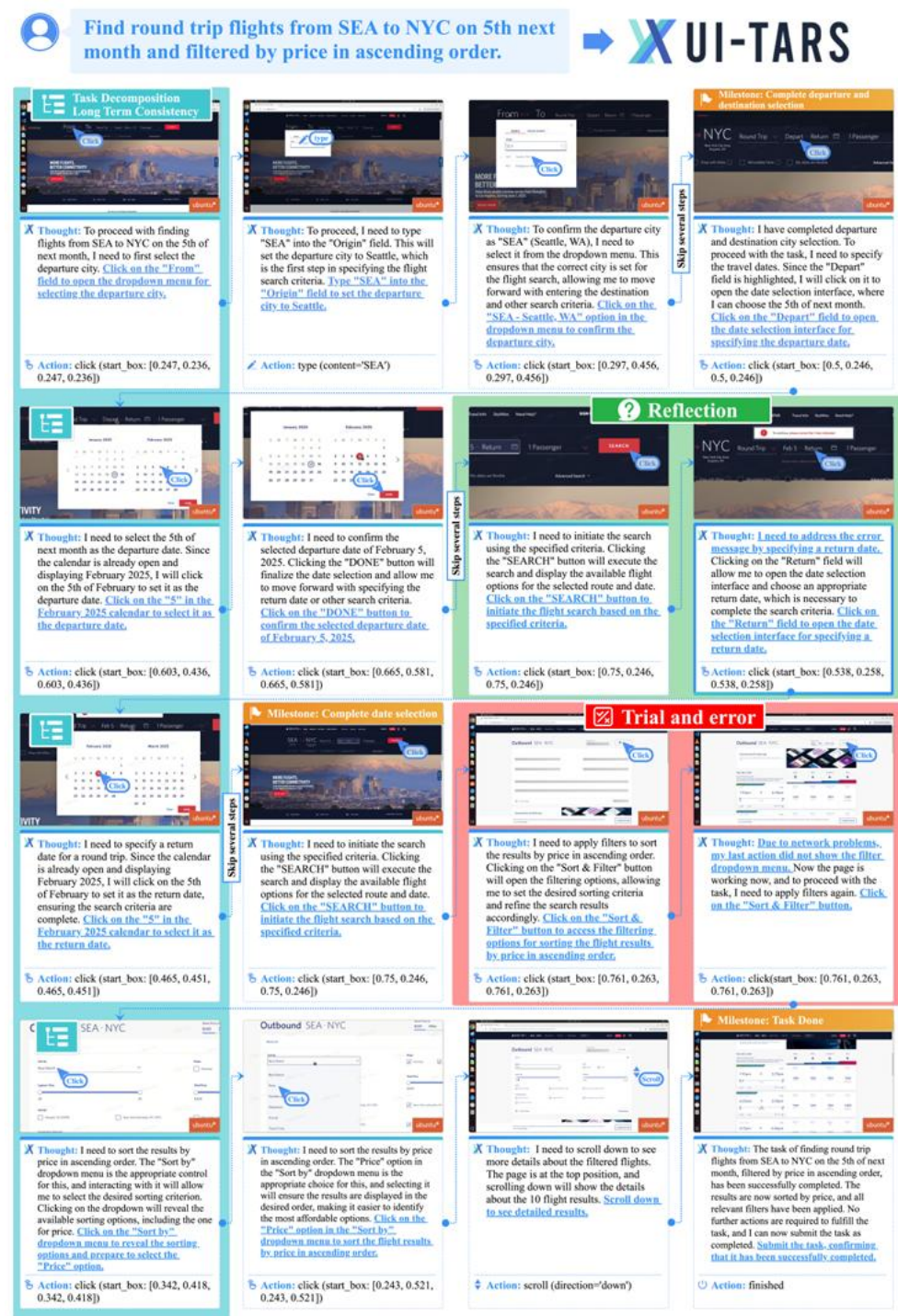
→ What safeguards and monitoring mechanisms can you implement?

Computer-Using Agents (CUA)

- Capabilities that enable AI agents to use computers like humans – look at screen, move cursor, click buttons, type text, etc.
- Examples:
 - [Computer Use Tool in Claude 3.5, 3.7 & 4 models](#)
 - [OpenAI Operator](#)
 - [Browser-Use](#) (Open-Source framework to connect agents with browser)
 - [Manus AI](#)
 - [UI-TARS](#) (by ByteDance)



Ref: [UI-TARS: Pioneering Automated GUI Interaction with Native Agents](#)



THANK YOU!



QUESTIONS?